

Fluffy Plushies Webshop



Hero Englund (SirMeows)

2026-06-01

GitHub: <https://github.com/WebSecDetectives/fluffy-plushies-webshop>

Deployment: <https://shop.starlords.eu>

List of Figures

Table 1	Minimum Application Features and Coverage	3
Table 2	Coverage of the Exam-Mandated Security Topics	3
Figure 1	Hosted Deployment Topology and Trust Boundaries	5
Figure 2	Authentication and Authorization Flow	6
Table 3	STRIDE Threats and Mitigations	7
Table 4	Derived Security Requirements and Coverage	7
Figure 3	Layered Request Gates for a Write	9
Figure 4	Secrets Provisioning Flow of the Deployed Application	14

List of Appendices

A Access Control Matrix	19
B IdentityService Test Suite	20
C Remaining Risks and Future Work	21
D Content Security Policy	22
E Dependency / Dependabot Remediation	23

Table of contents

1. Introduction	2
1.1. Project Description	2
1.2. Scope	2
1.3. Assumptions and Constraints	2
1.4. Requirements	3
1.5. Tech Stack	4
2. Threat Model and Architecture	5
2.1. System Overview and Trust Boundaries	5
2.2. Data Flow Diagram	6
2.3. Assets, Actors and Attack Surface (STRIDE)	6
2.4. Derived Security Requirements	7
3. Authentication, Authorization and Session Management	8
3.1. Authentication and Sessions	8
3.2. Token Lifetime, Logout and Revocation	8
3.3. Role-Based Access Control	8
3.4. Object-Level Authorization	8
4. Security Analysis and Mitigations	9
4.1. Input Validation	9
4.2. SQL and Command Injection	10
4.3. Cross-Site Scripting (XSS)	10
4.4. Cross-Site Request Forgery (CSRF)	11
4.5. File Upload Security	11
4.6. XXE, Serialization and Deserialization	12
4.7. Client-Side Manipulation	12
5. Framework-Provided Security and Dependency Management	13
5.1. Spring Security and Angular Built-in Protections	13
5.2. Automated Security Testing in CI (SAST / DAST)	13
5.3. Dependency and Supply-Chain Management (SCA)	13
6. Deployment, Secrets, Risk and Future Work	14
6.1. Secrets and Configuration Management	14
6.2. Infrastructure and Network Security	15
6.3. Logging Hygiene	15
6.4. Remaining Risks and Future Work	15
7. Conclusion	16
8. Glossary	17
9. References	18
A Access Control Matrix	19
B IdentityService Test Suite	20
C Remaining Risks and Future Work	21
D Content Security Policy	22
E Dependency / Dependabot Remediation	23

1. Introduction

1.1. Project Description

Fluffy Plushies is a small Danish e-commerce company with eight employees. It runs an online marketplace for plush toys and began as a collaboration between three makers and importers. The company is moving from selling its own stock to operating a platform that other independent makers sell through. A maker registers as a merchant and lists and manages its own products under a shared storefront. The platform provides the hosting, the customer accounts, and the access-control layer that keeps each merchant's catalogue separate. The longer-term goal is to give each merchant a fully hosted webshop on this shared platform.

The platform has three kinds of users. Customers browse the public catalogue, register an account, and review the products they can see. Merchants manage their own products and decide which are public and which stay private while in preparation. An administrator oversees the system and creates merchant accounts. The proof of concept (PoC) implements these roles and the boundaries between them. The rest of the report examines those boundaries from a security perspective.

1.2. Scope

The analysis covers three components. IdentityService handles registration, login, password storage, and user accounts. InventoryService handles products, images and reviews as well as their ownership and visibility. The Angular frontend handles session storage, role-based display, and the traffic between the browser and the two services. Together these contain the attack surface the report audits: authentication, authorization, input validation, stored user content, file upload, and session handling.

The repository derives from an earlier multi-service version written by the author and three others for a previous course [1]. The backend analysed here, IdentityService and InventoryService, was written by the author. This includes the authorization model and image-upload handling that the report examines most closely. The frontend was also primarily written by the author. Its initial setup and page layout were done by a co-author [2]. The two other services from the earlier iteration (OrderService and EmailService) remain in the repository but are out of scope. Docker Compose profiles keep them from starting in development, and the deployed version leaves them out entirely. That earlier iteration coordinated services over RabbitMQ messaging. The current design uses REST only, and the deployed system accepts REST calls alone. Legacy message-handling code remains in IdentityService but is unreachable in the deployment. Whether a final system stays REST-only or also offers messaging is a pending decision, recorded as future work (Section 6.4). Payment processing is not implemented. These exclusions keep the report on the parts of the system that are built, running, and demonstrable.

The Snapshot and Tombstone patterns (version snapshot and soft delete) responsible for the repudiation defence are implemented in IdentityService. A production version of the application would implement them across all services. For the purposes of this PoC, the patterns are considered adequately demonstrated.

1.3. Assumptions and Constraints

The application is a PoC, not a production system. The aim is to expose and analyse a realistic attack surface rather than to ship a complete shop. Feature breadth is therefore limited, and some operational hardening is left as future work (Section 6.4). Considering that the brunt of the work is done by a single author, security-focused design, analysis, and demonstration are prioritised over completeness, feature richness, or UX/UI design.

The analysis is predicated on the following assumptions: 1) The deployment platform is trusted and is treated as an external provider (Section 6.2). 2) An administrator has broad authority over accounts and products by design, so the model defends against ordinary users, merchants, and outside attackers, not against a malicious administrator or a compromised server. 3) The JWT signing secret is assumed to stay secret. It is shared between the two backend services, so one issues tokens the other verifies. 4) Demonstration data, the default accounts and seeded products, exists only under a dedicated demo profile that would not be used in a real deployment.

1.4. Requirements

The exam defines two groups of requirements. Table 1 lists the minimum application features and where each is met. Table 2 lists the security topics the project must address and where each is covered. The mandated topics are a fixed external checklist. The report's own security requirements are not taken from that list. They are derived from the system's assets and threats in the threat model (Table 4), and every mandated topic maps onto one of them. The full per-endpoint authorization rules are in the access control matrix in Appendix A.

ID	Feature	Implementation	Covered in
FR-1	Multilevel login, backend authentication	JWT login, roles USER/MERCHANT/ADMIN	Section 3.1, Section 3.3
FR-2	New user registration	Self-service customer signup (POST /users)	Section 3.1
FR-3	Session storage	Signed JWT in localStorage, sent as Bearer header	Section 3.1, Section 4.4
FR-4	List of items created by users	Merchant-owned products in InventoryService	Section 3.4
FR-5	Public / private item visibility	Visibility enum, enforced server-side	Section 3.4
FR-6	Admin can see everything	Admin bypasses visibility filters	Section 3.4
FR-7	Adding data to an item	Product reviews	Section 3.4
FR-8	File upload for images	Product image upload, validated and re-encoded	Section 4.5

Table 1: Minimum Application Features and Coverage

ID	Topic	Covered in
ST-1	SQL injection	Section 4.2
ST-2	Command injection	Section 4.2
ST-3	Cross-site scripting (XSS)	Section 4.3
ST-4	Cross-site request forgery (CSRF)	Section 4.4
ST-5	XML external entity (XXE)	Section 4.6
ST-6	Serialization / deserialization	Section 4.6
ST-7	Client-side manipulation	Section 4.7
ST-8	Weak or missing server-side validation	Section 4.1
ST-9	Session IDs and CSRF tokens in the right place	Section 3.1, Section 4.4
ST-10	Framework-provided security (explained)	Section 5
ST-11	Password hashing	Section 3.1
ST-12	Transport Layer Security (TLS)	Section 6.2
ST-13	Uploaded file validation and storage	Section 4.5

Table 2: Coverage of the Exam-Mandated Security Topics

1.5. Tech Stack

The frontend is an Angular single-page application. The two backend services, IdentityService and InventoryService, are JDK24, Spring Boot 4 applications built on Spring Security 7, and both persist to MySQL. The full source is available at [2].

In development the services run under Docker Compose, with nginx serving the frontend and proxying API calls to the backends.

The deployment runs on a Kubernetes cluster of Debian nodes provisioned with an Ansible IaC solution. Its images are built by Woodpecker and pulled from a self-hosted Zot container registry. Secrets are held in OpenBao key vault. There the frontend and backends are served same-origin behind a Traefik ingress over TLS. The two environments therefore differ in transport security and origin handling, examined in Section 4.4 and Section 6.2.

2. Threat Model and Architecture

2.1. System Overview and Trust Boundaries

Figure 1 shows the deployment topology. The browser reaches the system through a single origin. A Traefik ingress terminates TLS and routes by path. The root path serves the webshop frontend, /identity reaches IdentityService, and /inventory reaches InventoryService. Because the frontend and both APIs share one origin, the browser makes no cross-origin request in the deployment. The backend ports are not exposed outside the cluster, so the ingress is the only entry point.

The primary trust boundary lies between the browser and the server. Everything the browser sends is untrusted. It is validated and authorized on the server before it has any effect. A second boundary separates the two backend services from the data they hold. IdentityService signs a JSON Web Token (JWT) and InventoryService verifies it with the same shared secret, so InventoryService authorizes a request on its own without calling back to IdentityService. The MySQL databases sit behind the services and are never reached directly from outside the cluster. These boundaries are where the security requirements derived later in this chapter apply.

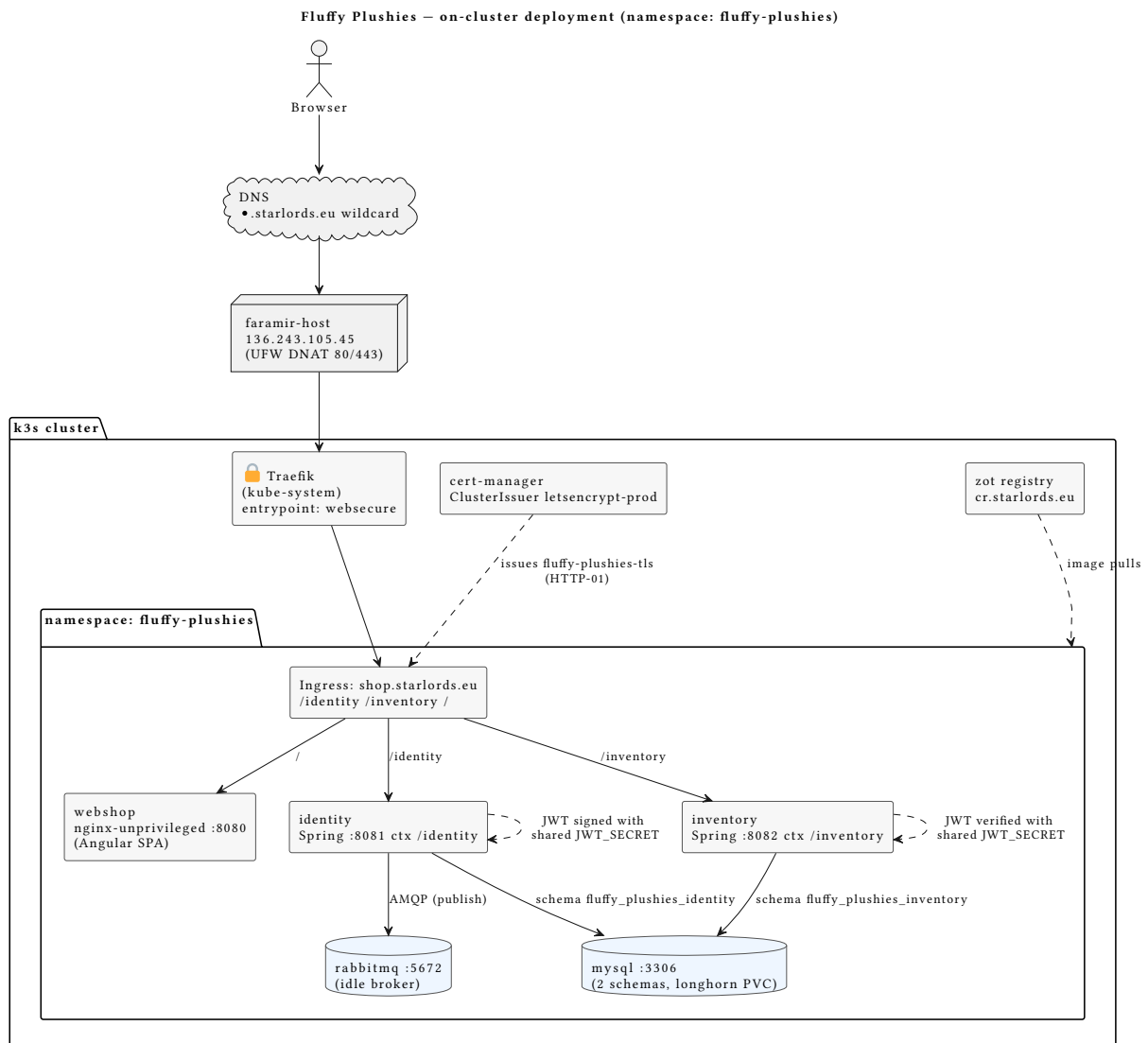


Figure 1: Hosted Deployment Topology and Trust Boundaries

2.2. Data Flow Diagram

Figure 2 traces the two flows that matter for authorization. In the first, the browser sends credentials to IdentityService, which verifies the password against its BCrypt hash and returns a signed JWT carrying the subject, role, and expiry. The token is stored in localStorage. In the second, the browser attaches that token as a Bearer header on each request to InventoryService. InventoryService verifies the signature and expiry with the shared secret, derives the caller from the token, and only then applies the role and ownership checks before reading the allowed rows from MySQL. The one point where data crosses the trust boundary is the Bearer request. Only the signed token travels, and a request without a valid one is rejected before any handler runs.

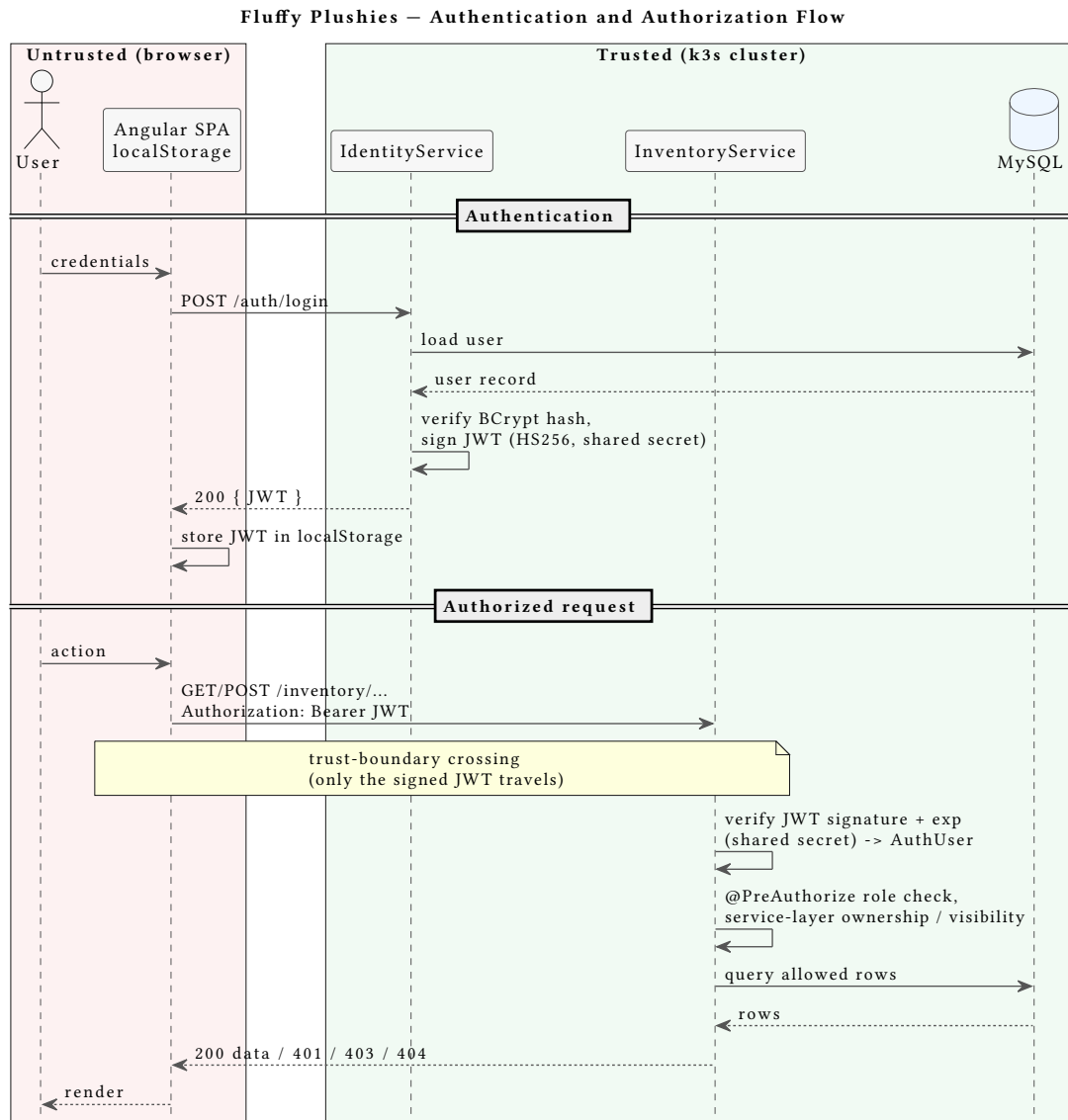


Figure 2: Authentication and Authorization Flow

2.3. Assets, Actors and Attack Surface (STRIDE)

The assets worth protecting are the user credentials and the JWT signing secret, user PII, the product catalogue, and the uploaded images. The actors are anonymous visitors, authenticated USER, MERCHANT, and ADMIN accounts, and an external attacker. The roles and what each may do are listed in Appendix A. Table 3 classifies the threats to these assets with STRIDE and points to where each is addressed.

Category	Threat in this application	Mitigation
Spoofing	Logging in as another user or forging a session	BCrypt-checked login and an HMAC-signed JWT verified on every request (Section 3.1)
Tampering	Altering request bodies, ownership fields, or stored content	Server-side validation (Section 4.1) · Ownership set from the JWT, not the body (Section 3.4)
Repudiation	Denying an action such as a posted review or an account change	Author and owner recorded server-side · IdentityService retains soft-deleted rows and version snapshots for later review
Information disclosure	Leaking PII, private items, or password hashes	404-vs-403 existence hiding (Section 3.4) · Hashes never serialised · TLS in transit (Section 6.2)
Denial of service	Decompression-bomb uploads or request flooding	Upload size and dimension caps (Section 4.5) · Rate limiting at the deployed edge (Section 6.2)
Elevation of privilege	A USER acting as a MERCHANT or ADMIN	Role taken from the signed JWT and checked with @PreAuthorize, re-verified server-side (Section 3.3, Section 4.7)

Table 3: STRIDE Threats and Mitigations

2.4. Derived Security Requirements

The STRIDE analysis [3] and the OWASP Top 10 [4] yield the security requirements in Table 4. Each requirement has an identifier and is traced to its mitigation in Section 4, so every control answers a stated requirement rather than standing on its own.

ID	Requirement	Met in
DSR-1	Credentials are verified server-side, and every request carries a signed token verified before processing.	Section 3.1
DSR-2	Every action is authorized server-side from the token, with ownership and visibility enforced per instance.	Section 3.3, Section 3.4
DSR-3	All client input is validated at the server trust boundary before use.	Section 4.1
DSR-4	Owner, role, and identity are taken from the token, never from the request body.	Section 3.4
DSR-5	Password hashes, private items, and PII are not exposed to unauthorized callers, and existence is hidden where needed.	Section 3.4
DSR-6	Uploaded images are validated by content and re-encoded before storage and serving.	Section 4.5
DSR-7	Resource exhaustion is bounded by upload caps and edge rate limiting.	Section 4.5, Section 6.2
DSR-8	Security-relevant actions are attributable to an identity, and identity data is retained for later review.	Section 1.2 (PoC)

Table 4: Derived Security Requirements and Coverage

3. Authentication, Authorization and Session Management

3.1. Authentication and Sessions

User passwords are never stored in plaintext. IdentityService hashes them with BCrypt through Spring Security's DelegatingPasswordEncoder, which prefixes each hash with its algorithm so the scheme can be upgraded later without invalidating existing accounts [5]. Login verifies the submitted password against the stored hash, and only a correct match yields a token.

On success IdentityService issues a JWT signed with a shared secret using HMAC [6], [7]. The token carries the user id, role, and an expiry, and nothing sensitive. The Angular client stores it in localStorage and attaches it as an Authorization: Bearer header on requests to the two backends. There is no server-side session. Both services run stateless (SessionCreationPolicy.STATELESS) and re-verify the token's signature and expiry on every request, so the signed token is itself the session identifier. Cookies are deliberately avoided. With no cookie there is no credential the browser sends automatically, which removes the CSRF surface (Section 4.4). The trade-off is that a token in localStorage is readable by JavaScript, so the cross-site scripting defences in Section 4.3 are vital.

3.2. Token Lifetime, Logout and Revocation

Authentication is stateless, so there is no session to destroy on the server. Logout is therefore a client-side action. The Angular client discards the token from localStorage and resets its state. The token itself stays cryptographically valid until it expires, as there is no server-side denylist to reject it earlier.

The mitigation for a stolen or lingering token is a short lifetime. The expiry is set to one hour (jwt.expirationMs). Because there is no refresh token, this expiry is also the full session length, so shortening it further would only log users out mid-use for little gain in a PoC. Immediate revocation would need either a refresh token plus a revocable server-side store, or a denylist keyed by the token's unique identifier (jti) claim and checked on every request. Both give up part of the stateless design and are documented as future work rather than implemented (Section 6.4).

3.3. Role-Based Access Control

Role and authority are taken from the signature-verified JWT and checked at the endpoint with @PreAuthorize, so every request is authorized server-side before any handler runs. The three roles are USER, MERCHANT, and ADMIN. POST /items is MERCHANT-only, so even an administrator cannot create a product. The full per-endpoint breakdown is given in Appendix A.

3.4. Object-Level Authorization

Role checks in the controller with @PreAuthorize do not address resource ownership, so instance-level ownership and visibility are enforced in the service layer, where the row's owner is known. Items are PUBLIC or PRIVATE, and reviews inherit their item's visibility. A resource the caller may not even see (a private item that is not theirs) returns **404**, hiding its existence. **403** is reserved for when the resource is visible but the action is not permitted, e.g. to prevent a merchant from editing another merchant's public item. The complete matrix, including these 404-vs-403 outcomes, is in Appendix A.

4. Security Analysis and Mitigations

The controls in this chapter do not act in isolation. Every request passes through a sequence of gates, each returning a specific HTTP status when it fails. Only a request that clears all of them reaches the business logic. Figure 3 shows this for a write to an item. The first gates are enforced by the framework as configured for this project, namely authentication, bean validation, and method-level role checks. The final gate is application code that decides ownership and visibility. The sections that follow examine each control in turn.

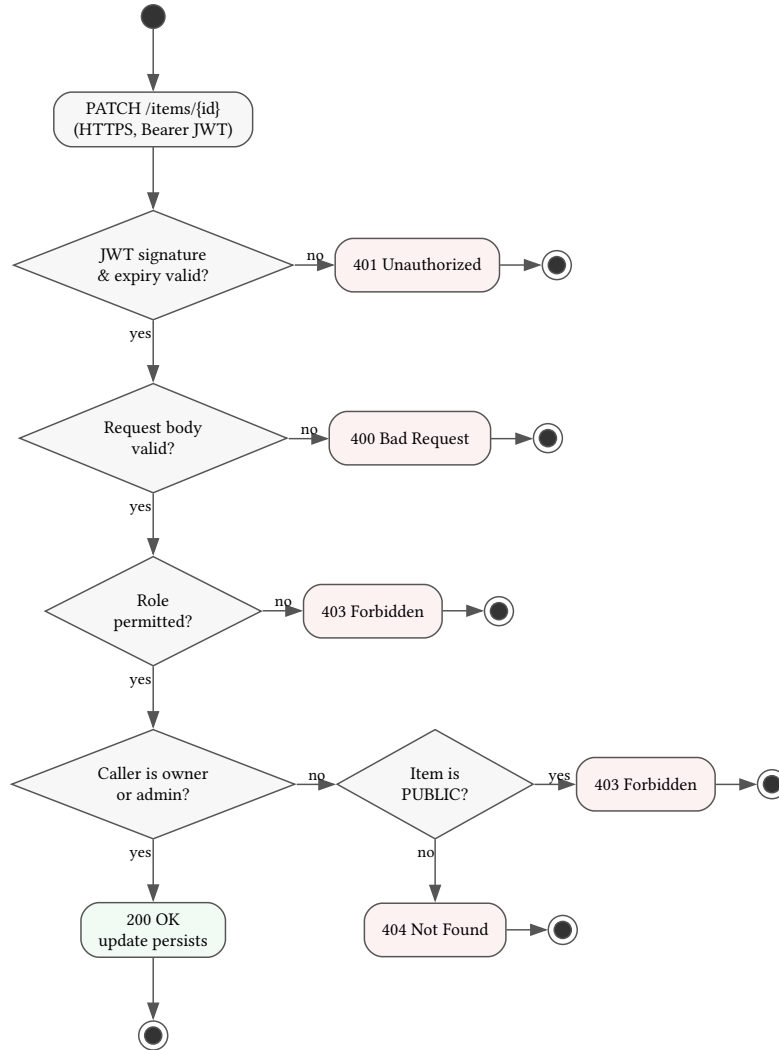


Figure 3: Layered Request Gates for a Write

4.1. Input Validation

Without server-side validation a client could submit malformed, oversized, or unexpected fields and have them persisted. Validation runs in two layers [4]. Every request body is a dedicated DTO annotated with Jakarta Bean Validation and checked by `@Valid` on the controller, which rejects bad input with 400 Bad Request before any business logic runs. The entities carry their own constraints as well (`@NotBlank`, `@Min`, column length limits), so the database is protected even on paths that do not pass through a request DTO, such as the data seeder.

Each operation has its own DTO. The create DTOs require every field, while the update DTOs make every field optional and apply a value constraint only when the field is present. A partial update therefore leaves omitted fields unchanged. Neither kind of DTO carries an `id` or an `owner` field, so a client cannot set `merchantId`, `reviewerId`, or a record `id` through the body. This closes the

mass-assignment and over-posting risk. The partial update also boxes `stock (int)` to `Integer`, because an omitted primitive would default to zero and reset the stock count.

Two fields show the approach on more complex input. The email address is validated with Jakarta's `@Email`, which checks the format only. This is deliberate. Full email validation is impractical, and the address serves only as a login identifier whose uniqueness the database enforces. Password strength is checked by a custom `@StrongPassword` constraint backed by the Passay library [8], requiring at least twelve characters with upper- and lower-case letters, a digit, a special character, and no whitespace. Reusing an established library here, rather than a hand-crafted regular expression, is the appropriate choice for a rule set that is easy to get subtly wrong. A complex regular expression would also be harder to read and could be slow enough to exploit as a DoS vector.

The rules were exercised by sending malformed and over-posted bodies through the Postman collection and confirming each returns `400` with a generic message instead of being persisted. One known behaviour remains. For endpoints guarded by method-level `@PreAuthorize`, Spring validates the body before the security check, so an unauthorized caller who sends an invalid body sees the `400` before the `403`. Because validation depends only on the request body and runs before the resource is looked up, the `400` reveals nothing about the target item's existence or ownership. It cannot be used to probe resources.

4.2. SQL and Command Injection

SQL injection arises when untrusted input is concatenated into a query. This is not a concern here, because the application never builds SQL as strings. All persistence goes through Spring Data JPA repositories using derived query methods such as `findByVisibilityOrMerchantId`, which Hibernate compiles to prepared statements with bound parameters. There are no `@Query` annotations, no native queries, and no `JdbcTemplate`. The only direct use of the `EntityManager` toggles Hibernate's soft-delete filter and builds no query text. Input is always bound as a parameter and never parsed as SQL, so an injection has nowhere to land.

Both manual tests and automated pipeline scanning cover this. A Postman injection folder fires SQL- and command-injection payloads at the login and item endpoints and asserts each is safely rejected or stored as inert data. CodeQL runs as static analysis and flags string-concatenated queries [9], and the OWASP ZAP full scan actively fires injection payloads at the running endpoints [10]. Both run as jobs in the project's GitHub Actions workflow, on every push to the main branch and on pull requests. Any native query added later must keep using bound parameters, since the ORM's protection does not extend to hand-written SQL.

Command injection requires handing untrusted input to an operating-system command, which the application never does. Neither service uses `Runtime.exec`, `ProcessBuilder`, or any shell invocation, confirmed by searching the codebase and by CodeQL. There is no command-injection surface to mitigate [4]. If an external program would be needed later, its arguments should be passed as an explicit list rather than a shell string, so nothing is interpreted by a shell.

4.3. Cross-Site Scripting (XSS)

User-generated content such as usernames, reviews, and item names is shown back to other users, which is the classic stored-XSS vector. The defence is Angular's default output encoding. Values placed through interpolation or property bindings are inserted as text and not parsed as HTML, so a stored `<script>` or `<img onerror=...>` renders as literal characters. The dangerous sinks such as `[innerHTML]` are run through Angular's sanitizer, and templates are compiled ahead of time, so user data can never become template code. These are framework mechanisms, covered in full in Section 5. They stay in place as long as the application does not opt out of them, for instance by

calling Angular's `bypassSecurityTrust*` methods or writing to the DOM directly through `innerHTML`, `nativeElement`, or `document`.

An audit of the frontend found none of these opt-outs in application code, so every value reaches the page through Angular's escaping. This was confirmed by manual testing. Storing `<img src=x onerror=alert(document.domain)>` and `<script>alert(1)</script>` as a review, as an item name, and as a username produced literal text on every page and no dialog. The stored database row held the payload unchanged, which confirms the defence is output encoding at render time, not input stripping. The OWASP ZAP full scan also probes the running site for reflected and stored XSS [10].

Output encoding at the point of rendering is the standard defence and is far harder to get wrong than manual escaping [11]. The session token lives in `localStorage` and is readable by JavaScript, so any XSS that did execute could exfiltrate it. This is the trade-off of the no-cookie design discussed in Section 4.4. A Content Security Policy is deployed in report-only mode as a second layer (Appendix D), with full enforcement planned for a later iteration. The protection holds only as long as no `bypassSecurityTrust` call or DOM-touching dependency is introduced.

4.4. Cross-Site Request Forgery (CSRF)

Cross-site request forgery works because a browser attaches a session cookie to a request automatically, including a request made by a malicious page on another origin. This application has no session cookie. The session is a JWT held in `localStorage` and attached as an `Authorization: Bearer` header only by the application's own code. A forged cross-site request cannot read that token or set the header, so it arrives unauthenticated and is rejected with 401. A CSRF token is therefore not used, because there is no automatically attached credential for a forged request to abuse. Spring Security's CSRF protection is disabled by design. This is the correct configuration for a stateless Bearer API rather than an omission [12].

This was demonstrated with a PoC page, served from a different origin, that auto-submits a cross-site POST `/inventory/items` while a user is logged in. The backend answered 401, and the request carried no cookie and no `Authorization` header, both visible in the browser's network trace. The 401 response additionally sets `X-Frame-Options: DENY`, so the attacker page cannot even render the result in a frame.

CORS does not change this. CORS controls whether a page may read a response, not whether a credential is attached to a request, so it cannot open a forgery path either way. The deployed demo is served same-origin behind the proxy with no backend CORS, so cross-origin reads are refused outright. The wildcard origin seen in development is only a local convenience for tooling such as Postman, and is not part of the deployed configuration. The mirror image of the no-cookie design is that the token in `localStorage` is exposed to XSS, which is addressed in Section 4.3.

4.5. File Upload Security

File upload is a high-risk feature because the client controls the bytes and the declared type. The pipeline trusts neither [13]. The accepted format is decided from the file's magic bytes, not its `Content-Type` or filename, and only JPEG and PNG pass. The dimensions are read from the header before any pixel is decoded, and rejected above 4096 pixels. That blocks a decompression bomb. The upload is then re-encoded to PNG with the Scrimage library [14]. Re-encoding is the core control. The stored bytes are freshly generated pixels, so anything hidden in the original, whether EXIF metadata or a polyglot payload, never reaches the database. A file that cannot be decoded is rejected.

Storage and serving close the rest of the surface. The bytes are stored as a database BLOB keyed by the item's UUID, in their own table, with no filename and no filesystem path. Path traversal therefore has nowhere to act. Uploading is gated to MERCHANT and ADMIN at the endpoint, and then to the item's owner in the service, so a merchant can only attach images to its own products.

Reading an image runs the same visibility check as the item itself. The image of a private item the caller cannot see, and the image of an item that does not exist, both return 404. Responses carry the stored `image/png` content type and the `X-Content-Type-Options: nosniff` header from Spring Security's defaults. Because the stored bytes are always a re-encoded PNG, there is no HTML or script for a browser to misinterpret.

The size is capped at 2 MB by the multipart configuration, and the proxy in front of the service is set to allow that body size. The controls were tested through the upload flows. A text file, or a text file renamed to `.png`, is rejected by the magic-byte check. The same request sent directly through Postman, bypassing the frontend, is rejected in the same way, which shows the rule is enforced on the server and not only in the browser. There is no anti-malware scan of uploads, although re-encoding removes most of that risk. Additionally, storing image bytes in the database does not scale well. In production, the answer would be to use object storage with signed URLs.

4.6. XXE, Serialization and Deserialization

XML external entity attacks require an XML parser. This system has none. Both services expose JSON-only REST APIs, and no endpoint or library parses untrusted XML. A search of both codebases for XML parsers, such as `DocumentBuilder`, `SAXParser`, `JAXB`, and Jackson's `XmlMapper`, returns nothing. XXE has no attack surface here [15].

Deserialization is handled by Jackson, which binds incoming JSON onto known DTO classes. The dangerous Jackson feature, polymorphic or default typing, is not enabled. Nothing in the code calls `activateDefaultTyping` or annotates a type with `@JsonTypeInfo`. Without it, a request body cannot choose the class to instantiate. The gadget-chain deserialization attacks seen in Java therefore do not apply. The request DTOs are plain data holders with no behavior and no fields a client should not set (see Section 4.1).

The one place that parses a security-sensitive token is the JWT filter, and it fails closed. A malformed, expired, or wrongly signed token is caught in the filter, logged at WARN by class name only and never as the token itself, and the request continues unauthenticated. The result is a clean 401, not a 500. This was confirmed during testing, where a tampered token produced a `MalformedJwtException` and a 401. These conclusions hold only while the API stays JSON-only and Jackson typing stays off. Adding XML parsing or enabling polymorphic typing later would reopen both.

4.7. Client-Side Manipulation

The client cannot be trusted to enforce anything, and the application does not ask it to. The Angular app hides controls by role and guards its routes, but that is user experience only. Every authorization decision is made again on the server, from the signature-verified JWT, never from anything the client stores or sends. Figure 3 traces this for a write. The role and ownership checks there run regardless of what the UI allowed (see Section 3.3 and Section 3.4).

Two manipulations show the boundary. Editing the role the frontend keeps in `localStorage` changes what the UI draws, not what the server enforces, because the role used for access decisions comes from the signed token. Tampering with the token itself breaks its signature, and the request is rejected with 401. This was confirmed by altering one character of the stored token. The second manipulation skips the UI entirely and calls the API directly with Postman, which reaches the same server-side checks. The Postman access-control tests use that path to verify each role's permissions independently of the frontend [4]. Client-side manipulation is assumed, and no security decision depends on the client.

5. Framework-Provided Security and Dependency Management

5.1. Spring Security and Angular Built-in Protections

The exam requires that framework-provided security be explained rather than assumed. Two frameworks do most of this work. Spring Security is secure by default. It authenticates every request in a filter chain before any controller runs, enforces roles through method security (@PreAuthorize), and sets defensive response headers without being asked, including X-Content-Type-Options: nosniff and X-Frame-Options: DENY. Those headers were visible on every response during the testing in Section 4.4. Its CSRF protection exists and is deliberately disabled, which is the correct choice for a stateless Bearer API and is argued in Section 4.4. Password hashing uses its DelegatingPasswordEncoder, covered in Section 3.1.

Angular does the equivalent on the client. It treats every interpolated value as untrusted and output-encodes it, runs the dangerous binding sinks through its DomSanitizer, and compiles templates ahead of time so user data cannot become template code. This is the main XSS defence, detailed in Section 4.3.

Relying on a framework is not free of risk. Both have CVE histories. Angular's sanitizer has had bypass advisories in older majors, and Spring Security has had its own. The defence is to stay on a current major version and to let dependency scanning catch regressions, which is the subject of the supply-chain section below [16], [17].

5.2. Automated Security Testing in CI (SAST / DAST)

Security testing runs in the GitHub Actions pipeline, per service, on every push to the main branch and on every pull request. Three scanners cover different ground. CodeQL performs static analysis over the Java source and flags taint and injection sinks [9]. The OWASP ZAP full scan drives the running service and probes it for injection, cross-site scripting, and missing headers [10]. Trivy scans the built container image for known-vulnerable packages. Each has a blind spot. Static analysis misses logic errors, dynamic scanning only sees the surface it can reach, and none of them can tell whether an authorization rule is correct.

Authorization correctness is proven by tests instead. IdentityService has a fully fledged Spring Boot integration test suite that runs through mvn verify and gates the pipeline. The test suite comprises 40 tests, many of them parameterized, for 111 test cases in total. It is built on the access control matrix (Appendix A) and covers registration, login, RBAC, password hashing, and JWT handling, with the full breakdown in Appendix B. InventoryService's access control is covered in the same spirit by the Postman access-control suite. These tests are the project's strongest evidence that authorization behaves as specified, because they assert the decisions a scanner cannot reason about.

5.3. Dependency and Supply-Chain Management (SCA)

Dependencies are tracked with Dependabot, and the clearest result is a contrast. Upgrading the two backend services to Spring Boot 4 and Spring Security 7 closed one critical and one high alert per service with no application-code change, just a version bump. The frontend, by comparison, carried 55 npm alerts. The cause is not effort but dependency-management style. Spring Boot's parent BOM pins the whole transitive graph to patched versions, so few CVEs ever surface. The npm ecosystem has no equivalent central pin, so transitive vulnerabilities accumulate. The right control therefore differs by ecosystem. On the backend it is to keep the framework BOM current and to centralize versions through POM properties. On the frontend it is to update the few direct dependencies and commit a refreshed lockfile, which cleared all 55 alerts. Trivy adds a third layer in CI by scanning the container images. The deployed container also pins its JDK base image to a digest rather than a version tag, so a tampered or replaced upstream image would have a different hash and would not be pulled. The closed alerts are listed in Appendix E.

6. Deployment, Secrets, Risk and Future Work

6.1. Secrets and Configuration Management

No secret is kept in source code, in a container image, or in git. In development the values live in a git-ignored `.env`, and only a `dotenv-template` with empty keys is committed. The Postman collection in the repository has every password and token blanked. In the deployment the secrets are held in an encrypted vault and injected into each pod as environment variables at runtime. Figure 4 shows the path. Ansible-encrypted secret files seed the OpenBao key vault, and a Kubernetes operator reads the values from there and mounts them on the services. The application only ever sees environment variables and has no knowledge of the vault. The JWT signing secret is delivered the same way and is shared between the two backends, so one signs tokens the other verifies (Section 3.1). The seeded demonstration accounts are created only under the demo profile, so a non-demo deployment starts with an empty database.

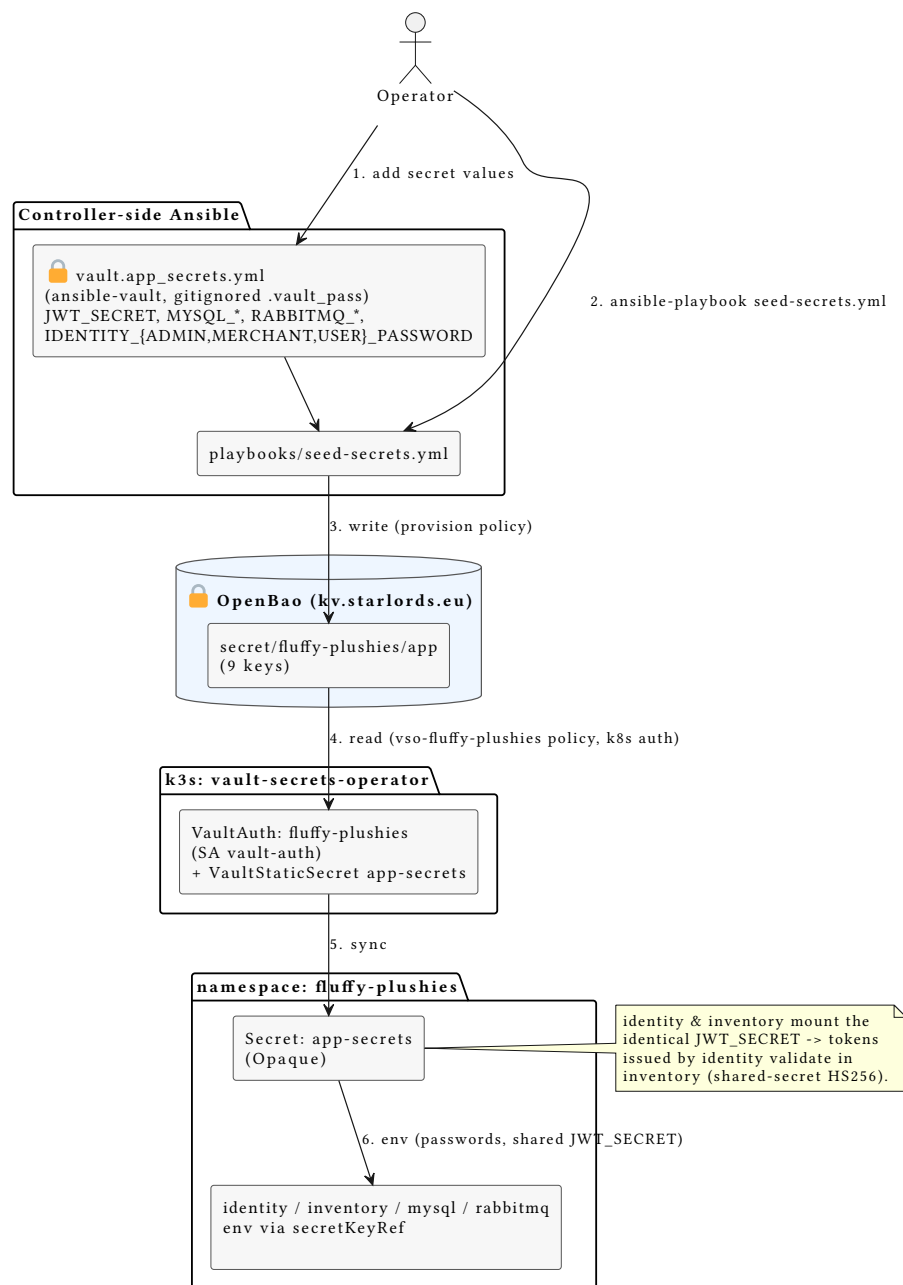


Figure 4: Secrets Provisioning Flow of the Deployed Application

6.2. Infrastructure and Network Security

The deployment is treated here as an external platform. The report states the security properties it provides, not its internals. Figure 1 shows the topology. Traffic reaches a single host and is routed into a Kubernetes cluster. TLS is terminated at the ingress, whose certificate for `shop.starlords.eu` is issued and renewed by a cluster certificate manager. The ingress routes by path: `/` to the frontend, `/identity` and `/inventory` to the two services. Without explicit CORS configuration the backend sends no CORS headers, so browsers block cross-origin reads. The MySQL databases are not reachable from outside the cluster. The containers are also hardened below the application. They run as a non-root user rather than root, which limits what a compromise inside a container could reach. The JDK base image is pinned to a digest, covered in Section 5. The edge accepts only HTTPS and redirects any HTTP request to it. The HSTS header, which makes the browser enforce that, is also set.

6.3. Logging Hygiene

Logging avoids leaking sensitive data. Secrets and tokens are never written to the log, and the JWT filter records only an exception class name when it rejects a bad token, never the token itself (Section 4.6). Service logs record identifiers rather than whole entity objects or request payloads, so a single log line cannot expose user data. Centralized log collection, retention, and alerting are left as future work, since the PoC runs as a small set of containers rather than a fleet.

6.4. Remaining Risks and Future Work

The application has known limitations, accepted on purpose rather than overlooked. Each one is paired with why it is accepted now and what would address it in Appendix C. None of them change the security model. They mark where a working PoC would be hardened toward production.

7. Conclusion

This project built a small but realistic web application, then analysed, tested, and hardened it. The minimum feature set is met in full (FR-1 through FR-8, Table 1). Every security topic the exam requires is addressed in Section 4, and each is tied to a requirement derived from the threat model rather than to the checklist alone (Table 4). Where a topic has no attack surface, such as XXE, that is demonstrated rather than claimed.

The security claims rest on evidence. User creation, authentication, and authorization are covered thoroughly by the IdentityService integration tests and the Postman access-control suite. The pipeline runs static analysis, dynamic scanning, and dependency and image scanning on every change. The manual attacks were carried out and behaved as the design predicts: a stored XSS payload, a CSRF attempt, and a tampered token.

All that being said, security is a moving target. The security decisions made here are documented explicitly, to help prevent later changes from introducing vulnerabilities in the system. The organisation responsible for governance must carry that knowledge forward, as the system grows.

8. Glossary

- BOM** Bill of Materials — a dependency manifest that pins versions. Spring Boot’s parent BOM keeps transitive dependencies on patched releases.
- BCrypt** Adaptive password-hashing function with a built-in salt and tunable cost, used to store user passwords.
- CORS** Cross-Origin Resource Sharing — browser rules that control which origins may read a response from another origin.
- CSP** Content Security Policy — an HTTP response header that restricts which scripts and resources a page may load or run.
- CSRF** Cross-Site Request Forgery — tricking a logged-in browser into sending an unwanted authenticated request.
- DAST** Dynamic Application Security Testing — testing that probes a running application from the outside (here, OWASP ZAP).
- Decompression bomb** A tiny compressed image that decodes into a huge bitmap, large enough to exhaust memory.
- DFD** Data Flow Diagram — a model of how data moves between components and across trust boundaries.
- DoS** Denial of Service — an attack that exhausts resources to make a service unavailable.
- DTO** Data Transfer Object — a request/response shape kept separate from persistence entities.
- EXIF** Exchangeable Image File Format — metadata embedded in an image file, such as camera settings, timestamps, and GPS location.
- Gadget chain** A deserialization attack that strings together existing classes whose side effects during deserialization lead to code execution.
- HMAC** Hash-based Message Authentication Code — a keyed hash used here to sign and verify the JWT with a shared secret.
- HSTS** HTTP Strict Transport Security — a response header telling the browser to use only HTTPS for a site, so it never sends a plain-HTTP request.
- IDOR** Insecure Direct Object Reference — accessing another user’s resource by guessing or altering its identifier.
- JWT** JSON Web Token — a signed token carrying identity and role claims, used here as the stateless session credential.
- ORM** Object-Relational Mapping — a layer that maps database rows to objects and generates parameterized queries (here, Hibernate via Spring Data JPA).
- PII** Personally Identifiable Information — data that identifies a person, such as name, email, or address.
- PoC** Proof of Concept — a working demonstration built to expose and analyse behaviour, not a production system.
- Polyglot** A file valid as two formats at once (e.g. a real image that is also a script), used to smuggle a payload past a type check.
- RBAC** Role-Based Access Control — permissions granted by role (USER, MERCHANT, ADMIN).
- SAST** Static Application Security Testing — analysis of source code without running it (here, CodeQL).
- SBOM** Software Bill of Materials — an inventory of the components and dependencies in a build.
- SCA** Software Composition Analysis — scanning dependencies for known-vulnerable versions.
- STRIDE** Threat taxonomy — Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege.
- XSS** Cross-Site Scripting — injecting scripts that execute in another user’s browser.
- XXE** XML External Entity — an attack abusing an XML parser’s resolution of external entities.

9. References

- [1] H. Engelund, K. Bremholm Mortensen, M. Nieman Madsen, and D. Causevic, “Fluffy Plushies Web Shop – Original DLS Project Source Code.” Accessed: June 08, 2026. [Online]. Available: <https://github.com/DLSHomies/FluffyPlushiesWebShop>
- [2] H. Engelund and M. A. Khan, “Fluffy Plushies Web Shop – Source Code.” Accessed: June 08, 2026. [Online]. Available: <https://github.com/WebSecDetectives/fluffy-plushies-webshop>
- [3] A. Shostack, *Threat Modeling: Designing for Security*. Wiley, 2014.
- [4] OWASP Foundation, “OWASP Top 10.” Accessed: June 08, 2026. [Online]. Available: <https://owasp.org/www-project-top-ten/>
- [5] OWASP Foundation, “Password Storage Cheat Sheet.” Accessed: June 08, 2026. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
- [6] M. Jones, J. Bradley, and N. Sakimura, “RFC 7519: JSON Web Token (JWT).” Accessed: June 08, 2026. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc7519>
- [7] jwt, “JJWT – Java JWT: JSON Web Token for Java and Android.” Accessed: June 08, 2026. [Online]. Available: <https://github.com/jwt/jjwt>
- [8] Passay, “Passay – Password Validation for Java.” Accessed: June 09, 2026. [Online]. Available: <https://www.passay.org/>
- [9] GitHub, “CodeQL – Code Scanning.” Accessed: June 08, 2026. [Online]. Available: <https://codeql.github.com/>
- [10] OWASP Foundation, “OWASP ZAP – The Zed Attack Proxy.” Accessed: June 08, 2026. [Online]. Available: <https://www.zaproxy.org/>
- [11] OWASP Foundation, “Cross Site Scripting Prevention Cheat Sheet.” Accessed: June 08, 2026. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html
- [12] OWASP Foundation, “Cross-Site Request Forgery Prevention Cheat Sheet.” Accessed: June 08, 2026. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html
- [13] OWASP Foundation, “File Upload Cheat Sheet.” Accessed: June 08, 2026. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html
- [14] S. Sam, “Scrimage – Immutable, Functional, Performant JVM Image Library.” Accessed: June 08, 2026. [Online]. Available: <https://sksamuel.github.io/scrimage/>
- [15] OWASP Foundation, “XML External Entity Prevention Cheat Sheet.” Accessed: June 08, 2026. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/XML_External_Entity_Prevention_Cheat_Sheet.html
- [16] VMware / Broadcom, “Spring Security Reference.” Accessed: June 08, 2026. [Online]. Available: <https://docs.spring.io/spring-security/reference/index.html>
- [17] Google, “Angular – Security.” Accessed: June 08, 2026. [Online]. Available: <https://angular.dev/best-practices/security>
- [18] Aqua Security, “Trivy – Comprehensive Security Scanner.” Accessed: June 08, 2026. [Online]. Available: <https://trivy.dev/>
- [19] GitHub, “About Dependabot Alerts.” Accessed: June 08, 2026. [Online]. Available: <https://docs.github.com/en/code-security/dependabot/dependabot-alerts>

A Access Control Matrix

Endpoint	Everyone	USER	MERCHANT	ADMIN	Notes
Identity Service					
POST /auth/login	Yes	Yes	Yes	Yes	Login
POST /users	Yes	Yes	Yes	Yes	Register normal user
POST /users/merchant	–	–	–	Yes	Admin creates merchant accounts
POST /users/admin	–	–	–	Yes	Admin creates admin accounts
GET /users	–	–	–	Yes	Admin lists all users
GET /users/{id}	–	Own	Own	Yes	Self or admin
PATCH /users/{id}	–	Own	Own	Yes	Self or admin
PATCH /users/{id}/password	–	Own	Own	Yes	Self or admin
DELETE /users/{id}	–	Own	Own	Yes	Self or admin
GET /auth/validate	–	Yes	Yes	Yes	Token validation
Inventory Service					
GET /items	Public	Public	Pub+own	All	List filtered to visible rows
GET /items/{id}	Public	Public	Pub+own	All	Hidden private → 404 (not 403)
GET /items/mine	–	–	Own	–	Merchant-only · admin uses GET /items
POST /items	–	–	Yes	No	MERCHANT only — admin is not a superuser · merchantId from JWT
PATCH /items/{id}	–	–	Own	Yes	Not-owned private → 404, public → 403
DELETE /items/{id}	–	–	Own	Yes	Same ownership rule as PATCH
GET /reviews/item/{itemId}	Vis	Vis	Vis	Yes	Reviews follow item visibility
POST /reviews/item/{itemId}	–	Vis	–	Yes	USER/ADMIN only · reviewerId from JWT
GET /reviews	–	–	–	Yes	Admin review moderation
GET /reviews/{id}	–	–	–	Yes	Admin review moderation
PATCH /reviews/{id}	–	Own	–	Yes	Author edits own · admin any
DELETE /reviews/{id}	–	Own	–	Yes	Author deletes own · admin any

Legend: Yes = allowed · – = denied · Own = own resource only · Public = public items only · Pub+own = public items plus own private · All = all items · Vis = only if the item is visible to the caller.

B IdentityService Test Suite

Area	Verified behaviour	Result
Registration and input validation	Valid registration body	200 OK
	Empty body, or a null or blank required field	400
	Username outside the allowed length	400
	Invalid email format vs. valid formats and boundary lengths	400 / 200
	Weak password on password change	400
Password storage	Stored password is the hash, not the raw input	hash ≠ plaintext
Login	Correct credentials	200 with token and expiry
	Wrong password	401
	Stale Bearer token on the public login endpoint	ignored, 200
Role-based access	Admin creates an admin or merchant account	200
	USER attempts admin or merchant creation	403
	Created merchant persists the MERCHANT role	role saved
Ownership and resource state	Self or admin updates or deletes a user	200 / 204
	Non-owner, non-admin update or delete	403
	Update or delete of an unknown user id	404
	Update of a soft-deleted user	423
Token rejection (JWT filter)	Missing, malformed, or wrong-scheme token	401
	Expired token	401
	Tampered signature	401
	Validly signed token for an unknown user	401

Status codes: 200 OK · 204 No Content · 400 Bad Request · 401 Unauthorized · 403 Forbidden · 404 Not Found · 423 Locked.

C Remaining Risks and Future Work

Limitation	Why it is accepted	Planned improvement
The session token in localStorage is readable by JavaScript	The XSS defences mitigate it, and a report-only Content Security Policy is deployed (Section 4.3)	Enforce the Content Security Policy (currently report-only)
No server-side token revocation	A one-hour token lifetime limits the exposure window (Section 3.1)	Refresh-token rotation with short-lived access tokens
Images are stored as database blobs	Acceptable at demo scale (Section 4.5)	Object storage with signed URLs, and malware scanning of uploads
Soft-delete and audit retention exist only in IdentityService	Deliberate scope limitation, shown end to end in one service	Extend to the other services
Logging is not centralized	The PoC runs as a few containers, not a fleet	Centralize collection, retention, and alerting
Leftover RabbitMQ message-handling code in IdentityService	Unreachable in the deployment, which accepts REST only	Remove it, pending a REST-only vs. messaging decision
EmailService carries unremediated dependency alerts	Out of SWD scope	Left to the owning service

D Content Security Policy

The deployed application sends a Content-Security-Policy-Report-Only header. The browser reports policy violations without blocking them, so the policy can be tuned against real traffic before it is switched to enforcing. The capture below shows the report-only header in effect on the deployed application.

X	Headers	Preview	Response	Initiator	Timing
▼ General					
Request URL		https://shop.starlords.eu/polyfills-5CFQRCPP.js			
Request Method		GET			
Status Code		● 200 OK (from memory cache)			
Remote Address		136.243.105.45:443			
Referrer Policy		strict-origin-when-cross-origin			
▼ Response headers					
Accept-Ranges		bytes			
Content-Length		34585			
Content-Security-Policy-Report-Only		default-src 'self'; script-src 'self'; style-src 'self' 'unsafe-inline'; img-src 'self' data: blob; connect-src 'self'; object-src 'none'; base-uri 'self'; frame-ancestors 'none'; form-action 'self'			
Content-Type		application/javascript			
Date		Wed, 10 Jun 2026 19:05:27 GMT			
Etag		"6a25bfe6-8719"			
Last-Modified		Sun, 07 Jun 2026 19:00:54 GMT			
Referrer-Policy		strict-origin-when-cross-origin			
Strict-Transport-Security		max-age=86400			
X-Content-Type-Options		nosniff			
X-Frame-Options		DENY			

E Dependency / Dependabot Remediation

Backend (Spring services) — Dependabot alerts closed. Both were closed per service by the Spring Boot 4 / Spring Security 7 upgrade, with no application-code change.

Package	Severity	Vulnerable	Closed by	GHSA
spring-security-web	critical	6.4.0–6.4.13	Spring Security 7.0.5	GHSA-mf92-479x-3373
spring-boot-devtools	high	3.4.0–3.4.15	Spring Boot 4.0.6	GHSA-56v8-86gj-66jp

Frontend (Webshop, npm) — 55 alerts closed, by cluster. The full per-alert list is in Documentation/DEPENDENCY_SECURITY.md and package-lock.json.

Cluster	Count	Worst	Fix
protobufjs (+ @protobufjs/utf8)	10	critical	firebase 12.14.0 via npm update
lodash	9	critical	removed unused firestore package, then npm update
@angular/core / compiler / common	7	high	npm update to 20.3.24
build and tooling transitives (vite, rollup, tar, glob, minimatch, qs, postcss, ...)	29	high	lockfile refresh via npm update